RL Navigation Agent

Project Report

Team Members:
Aman Malik(23k-0871)
Muhammad Ahsan Shykh(23k-0541)
Muhammad Maarij(23k-0527)

Submitted To: Sir Qasim Keeryo
Date: 5th June 2026

1. Introduction

This project presents an implementation of a Reinforcement Learning (RL) based navigation agent that learns to find an optimal path from a start position to a goal position within a dynamically generated grid environment. The system is built entirely in Python and combines core machine learning concepts with an interactive graphical user interface (GUI).

The agent uses the **Q-Learning algorithm** — a model-free, value-based reinforcement learning technique — to learn navigation behavior through repeated trial and error interactions with the environment. Over hundreds or thousands of training episodes, the agent progressively improves its decision-making, moving from random exploration to intelligent goal-directed navigation.

2. Objectives

- Implement a grid-based environment suitable for reinforcement learning experimentation.
- Train a Q-Learning agent capable of navigating from a start cell to a goal cell while avoidingobstacles.
- Provide a real-time visual interface displaying the learning process, agent behavior, andperformance statistics.
- Incorporate mechanisms that handle edge cases such as loop detection during agent execution.

3. Tools and Technologies

No external machine learning frameworks (such as TensorFlow or PyTorch) are used. The Q-Learning algorithm is implemented from scratch using NumPy arrays.

Component	Detail
Programming Language	Python 3
GUI Framework	tkinter (standard Python library)
Numerical Computation	NumPy
Randomness	random (standard Python library)
Concurrency	threading (standard Python library)
Timing	time (standard Python library)

4. System Architecture

The project is organized into three main classes, each responsible for a distinct layer of the system:

Layer	Class	Responsibilities
User Interface	GUI	Controls, Canvas, Stats, Buttons

Intelligence	QlearningAgent	Q-Table, epsilon-greedy, Bellman Update
Environment	GridEnvironment	Grid, Obstacles, Rewards, Step Logic

5. Component Details

5.1 GridEnvironment Class

This class models the RL environment, generating the world in which the agent operates and defining the rules of interaction.

Grid Generation: The grid is a 2D NumPy array of size N×N (default 10×10). Cells are initialized as free (0) or obstacle (1) at a configurable density (default 20%). Start (S) and Goal (G) cells are randomly assigned to distinct free cells.

Actions: The agent can take one of four discrete actions at each time step:

Action Code	Direction	Row Change	Column Change
0	Up	−1	0
1	Down	+1	0
2	Left	0	−1
3	Right	0	+1

Reward Structure:

Outcome	Reward
Goal reached	+10
Hit an obstacle	−5
Moved out of bounds	−2
Normal valid move	−1 − (0.05 × Manhattan distance to goal)

The Manhattan distance component is a form of *reward shaping*: it penalizes the agent more heavily for moving away from the goal, guiding it toward goal-directed paths even in early training.

5.2 QlearningAgent Class

This class implements the Q-Learning algorithm. The agent maintains a **Q-table** — a 3D NumPy array of shape (grid_size × grid_size × 4). Each entry Q[r][c][a] stores the estimated cumulative future reward for taking action *a* from cell (r, c).

Hyperparameters:

Parameter	Symbol	Value	Role
Learning Rate	α	0.15	Controls how much new information overrides old

Discount Factor	γ	0.99	Weighs importance of future rewards
Initial Epsilon	ε	1.0	Starting exploration rate (fully random)
Epsilon Decay	—	0.997	Multiplier applied to ε after each episode
Minimum Epsilon	ε_min	0.01	Floor value — agent always retains 1% exploration

Action Selection (ε-Greedy Policy): At each step, the agent either *explores* (random action, probability ε) or *exploits* (best Q-value action, probability 1−ε). As ε decays over training, the agent transitions from random exploration to learned exploitation.

Q-Value Update (Bellman Equation):

$$Q(s, a) \leftarrow Q(s, a) + \alpha \times [\, r + \gamma \times \max Q(s', a') - Q(s, a) \,]$$

Where the term in brackets is the *Temporal Difference (TD) error*. If the episode ends (goal reached), the target simplifies to just r.

5.3 GUI Class

The GUI class manages the entire user interface using tkinter. It ties together the environment and agent, handles user inputs, and visualizes learning in real time.

Settings Sliders:

Slider	Range	Default	Purpose
Grid Size	5 – 100	10	Sets the N×N dimensions of the grid
Episodes	200 – 5000	1000	Number of training episodes
Obstacles	5% – 50%	20%	Density of obstacles on the grid
Anim Speed	0.02 – 0.8	0.5	Delay in seconds between animation frames

Grid Visualization: Free cells are colored using a Q-value heatmap after training. Cells with higher Q-values (closer to the optimal path) appear warmer, visually illustrating what the agent has learned.

Element	Color	Description
Agent	Red circle	Current position of the agent
Goal (G)	Green	Target cell
Start (S)	Orange	Starting cell
Obstacle	Dark/Black	Impassable cell
Trail	Dark Red	Cells visited during the current run
Free Cells	Blue-to-warm gradient	Q-value heatmap after training

6. Training Process

- A new episode begins — the agent is placed at the Start cell.
- At each step, the agent selects an action using the ε-greedy policy.
- The environment processes the action and returns the next state, reward, and done flag.
- The agent updates its Q-table using the Bellman equation.
- This repeats until the agent reaches the Goal or exhausts the maximum step limit (grid_size² ×4).
- After each episode, epsilon is decayed.
- After all episodes complete, the final success rate is displayed and the agent is ready to run.

The maximum step limit per episode is set to **grid_size² × 4**, which is generous enough for the agent to explore the grid multiple times without running indefinitely.

7. Key Design Decisions

Distance-based reward shaping: Rather than a flat −1 per step, the penalty increases with Manhattan distance to the goal, helping the agent learn goal-directed tendencies faster.

High discount factor (γ = 0.99): A value close to 1 means the agent cares heavily about long-term reward — appropriate for navigation where the goal may be many steps away.

Epsilon decay of 0.997: A slow decay rate, ensuring meaningful exploration for a substantial portion of training and reducing premature convergence to suboptimal paths.

Thread-safe GUI updates: All UI mutations are passed through root.after() to ensure execution on the main thread, preventing race conditions.

Q-value heatmap rendering: Post-training, free cells are colored by their max Q-value, making the learned policy visually interpretable.

8. Limitations

- The Q-table is indexed by grid coordinates — the agent must retrain from scratch whenever thegrid is reset; learned policies are not transferable.
- The environment does not verify that a valid path from Start to Goal exists. In high-densityobstacle configurations, the goal may be unreachable.
- The loop-break mechanism during the Run phase is heuristic and may not always produceoptimal detour decisions.
- Scaling to very large grids (e.g., 100×100) increases the Q-table to 40,000 entries and maysignificantly slow training.

9. Possible Improvements

- Add pathfinding validation (e.g., BFS check) before training to ensure the goal is reachable.
- Implement Deep Q-Network (DQN) to replace the tabular Q-table with a neural network, enablinggeneralization across different grids.
- Save and load Q-tables to disk to allow resuming training or reusing learned policies.
- Add a real-time reward/episode graph to better visualize the learning curve.
- Introduce multiple agents or moving obstacles to increase task complexity.

10. Conclusion

This project successfully demonstrates the Q-Learning algorithm applied to a grid navigation problem with a fully interactive visualization. The agent begins with no knowledge of the environment and, through thousands of trial-and-error episodes, develops a policy capable of navigating from Start to Goal while avoiding obstacles.

The GUI makes the learning process interpretable — showing the Q-value heatmap, live statistics, and animated agent runs. The project covers fundamental RL concepts including Markov Decision Processes, the Bellman equation, epsilon-greedy exploration, and reward shaping, making it a comprehensive practical implementation of reinforcement learning from scratch.